**Tech Challenge - Fase 3 - FIAP**

**Curso:** IA PARA DEVS

**Projeto :** Assistente Médico Virtual Hospitalar

**Autor:** Victor Luiz Domingues RM: rm375278

**Grupo:** 38 (Entrega individual)

**Data:** 2026

**Vídeo:**

https://www.youtube.com/watch?v=MaZKLVIN1gc

**Repositório:**
https://github.com/victorldomingues/pos_fiap_ia_fase3_tech_challenge

**Notebook (Principal/Execução)**
https://github.com/victorldomingues/pos_fiap_ia_fase3_tech_challenge/blob/main/IADT%203%20-%20Victor%20Luiz%20Domingues.ipynb

**Relatório Técnico:**

https://github.com/victorldomingues/pos_fiap_ia_fase3_tech_challenge/blob/main/medical-ai-assistant/docs/relatorio_tecnico.md

**Instruções de execução:**
Acesse o Github siga as intruções de instalacao e execucção

https://github.com/victorldomingues/pos_fiap_ia_fase3_tech_challenge/tree/main

**Arquitetura:**
https://github.com/victorldomingues/pos_fiap_ia_fase3_tech_challenge/blob/main/medical-ai-assistant/docs/arquitetura.md

# Instalação

## Google Colab — primeiro acesso

O notebook é a forma principal de executar a demonstração. Ele precisa dos arquivos de **medical-ai-assistant/** no Google Drive, no caminho exato **/content/drive/MyDrive/fiap/fase3/medical-ai-assistant**.

**1. Importe e salve o notebook.** Abra o notebook do projeto no repositório e importe-o no Google Colab; use Arquivo → Salvar uma cópia no Drive para guardar sua cópia de trabalho.

**2. Configure a GPU.** No Colab, selecione Ambiente de execução → Alterar tipo de ambiente de execução → T4 GPU e salve. A GPU pode depender da disponibilidade e dos limites da sua conta; confirme no notebook que aparece uma GPU CUDA antes de treinar.

**3. Monte o Drive e copie a aplicação.** Execute a primeira célula do notebook para montar o Drive. Em uma célula temporária, clone o repositório e copie somente a pasta da aplicação para o caminho esperado:

```
from pathlib import Path
import shutil
!git clone --depth 1 \
 https://github.com/victorldomingues/pos_fiap_ia_fase3_tech_challenge.git \
 /content/pos_fiap_ia_fase3_tech_challenge

source = Path("/content/pos_fiap_ia_fase3_tech_challenge/medical-ai-assistant")
target = Path("/content/drive/MyDrive/fiap/fase3/medical-ai-assistant")
target.parent.mkdir(parents=True, exist_ok=True)
shutil.copytree(source, target, dirs_exist_ok=True)
assert (target / "src").is_dir(), f"Pasta da aplicação não encontrada em {target}"
```

Faça essa cópia inicial antes de executar a seção **0. Ambiente**. Ela verifica PROJECT_DIR nesse local e instala requirements-train.txt e requirements.txt. Se a pasta já existir no Drive, não repita a cópia sem antes preservar seus dados e artefatos; os arquivos correspondentes podem ser atualizados.

**4. Configure os Secrets (opcional).** Selecione o ícone de chave Secrets na barra lateral do Colab e crie:

**HF_TOKEN:** token de leitura do Hugging Face. O modelo Qwen e os datasets usados são públicos, então normalmente é possível baixar sem token; configure-o para autenticar-se no Hub se necessário. Habilite o acesso do notebook ao secret. Para disponibilizá-lo às bibliotecas Hugging Face, execute uma célula antes do download do modelo:

```
import os
from google.colab import userdata

hf_token = userdata.get("HF_TOKEN")
if hf_token:
   os.environ["HF_TOKEN"] = hf_token
```

**LANGSMITH_API_KEY:** chave da sua conta LangSmith. É opcional e só é necessária para tracing na seção 3.6; habilite o acesso do notebook ao secret. LANGSMITH_PROJECT não é segredo e, se não for definido, o notebook usa medical-ai-assistant.

> **CUIDADO:** Não cole tokens no código ou em células salvas. Sem LangSmith, a demonstração funciona normalmente; o tracing fica desligado. Mesmo quando habilitado, o projeto omite inputs e outputs dos traces.

## Instalação local/CLI

Para executar fora do Colab, use **Python 3.11+**. Na raiz do repositório, entre em medical-ai-assistant/; os comandos das seções locais seguintes devem ser executados nesse diretório, salvo indicação diferente.

```
cd medical-ai-assistant
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt        # inferência + testes
# opcional (treino/datasets públicos): pip install -r requirements-train.txt
cp .env.example .env                   # ajuste e carregue:
# set -a; source .env; set +a
```

# Execução

## Execução no Google Colab

Execute as células em ordem, começando pela seção **0. Ambiente**. Essa seção monta o Drive, valida o caminho dos arquivos e instala as dependências. A instalação e os downloads exigem conexão com a internet; em um runtime novo, a instalação de dependências se repete.

### Parte 1 — dados

Execute a preparação. Para usar o RAG público, na seção 1.2 mude **DOWNLOAD_PUBLIC = False** para **True** e execute a célula. Ela baixa amostras de PubMedQA e MedQuAD para data/raw/; LIMIT define o número máximo de registros de cada base. Depois execute 1.3, que prepara data/processed/public_sources.jsonl e os splits de treinamento. As bases são públicas, não curadas e podem exigir tempo e espaço de armazenamento; verifique licenças e termos. Elas são usadas no RAG, não como respostas para o fine-tuning.

### Parte 2 — fine-tuning

A seção 2.2 mantém **RUN_TRAINING = False** por padrão. Para treinar o Qwen2.5-3B-Instruct com QLoRA, confirme a GPU T4, execute a Parte 1 e então defina **RUN_TRAINING = True**. O primeiro treino baixa o modelo-base e pode levar um tempo significativo. O adaptador, tokenizer e metadados são gravados no Drive em medical-ai-assistant/artifacts/qwen-qlora-adapter.

Confira se já há um adaptador antes de treinar: **OVERWRITE = True** permite sobrescrever artefatos existentes. Para seguir o fluxo padrão até 3.3, é preciso ter um adaptador compatível: treine na primeira

execução ou reutilize um já salvo. Em runtimes posteriores, mantenha o treino desligado e valide o adaptador na seção 2.3, em vez de retreiná-lo.

### Parte 3 — RAG

**USE_PUBLIC_RAG = True** na seção 3.1 exige que as duas bases tenham sido baixadas e preparadas nas seções 1.2 e 1.3. Se não quiser usar fontes públicas, defina **USE_PUBLIC_RAG = False** antes de executar 3.1. A primeira indexação baixa o modelo multilíngue de embeddings e sincroniza protocolos e, se habilitado, as fontes públicas. Execute 3.1 antes de 3.2 e 3.3.

O índice Chroma fica em /content/medical-ai-assistant-index/chroma, armazenamento temporário do runtime; após reiniciá-lo, execute novamente 3.1 para reconstruir o índice a partir dos dados preservados no Drive. Não é necessário acionar rebuild=True como rotina.

### Demonstração

Depois de inicializar o assistente na seção 3.3, continue as partes desejadas. A Parte 6 executa os cenários automatizados e imprime cada histórico no output do notebook; a Parte 7 inicia o chat interativo. A seção 3.6 é opcional e só deve ser executada se LANGSMITH_API_KEY estiver configurada.

O Google Drive conserva os dados-fonte e o adaptador, mas não é um banco gerenciado. O índice em /content é efêmero por decisão: preserve data/protocols, os arquivos originais em data/raw/, o corpus processado e o adaptador no Drive. A disponibilidade de GPU e os limites de memória podem variar; se faltar memória, interrompa o treino e reduza a carga antes de tentar novamente.

## Execução local/CLI

```
python scripts/build_index.py  # ação explícita: indexa data/protocols no ChromaDB
python main.py --user usuario-01 --reviewer dra-demo
```

### Fluxos fictícios de médico/paciente

Fluxos fictícios de médico/paciente em SQLite (token administrativo de provisionamento deve vir do ambiente):

```
MED_DB_BOOTSTRAP_TOKEN='<segredo-local>' \
  python scripts/clinic_demo.py --provision-doctor
python scripts/clinic_demo.py
```

O fluxo pede o perfil quando não há sessão e valida o perfil real pelo token, não pela opção escolhida. Cadastros exigem confirmação de uso exclusivo de dados sintéticos; o token do paciente/profissional é exibido uma vez. Agendamento exige um médico provisionado e horário ISO-8601 com fuso.

### Testes automatizados da demonstração

Para percorrer automaticamente os fluxos da demonstração sem autenticação manual:

```
python scripts/clinic_demo.py --automated-tests
```

O modo executa seis cenários com SQLite temporário, cobrindo as quatro opções do paciente, as nove opções do médico, cadastro, isolamento da agenda e uma credencial incompatível. Para cada cenário, imprime as entradas sintéticas e a saída completa da CLI diretamente no console/output do Colab, com credenciais e dados pessoais redigidos. Não grava transcript em arquivo.

Usa respostas e embeddings determinísticos (sem baixar modelo nem acessar o banco persistente). Esse comando testa os casos de uso da CLI; a suíte unitária e de integração continua sendo executada separadamente com **python -m pytest**.

## Indexação e atualização de protocolos

O primeiro build_index baixa o modelo de embeddings (paraphrase-multilingual-MiniLM-L12-v2). Sem o índice, main.py falha com instruções claras (nunca indexa silenciosamente). Alterou os protocolos? Rode build_index.py de novo (atualização incremental por hash) ou --rebuild.

## RAG público acadêmico

No Colab, execute 1.2 e 1.3 com as duas bases, depois 3.1 com **USE_PUBLIC_RAG=True** e reinicialize o assistente em 3.3. As fontes públicas não curadas ficam em índice separado; cada resultado identifica dataset, registro e hash do conteúdo, não uma versão oficial. Não são protocolos institucionais nem evidência clínica validada. Similaridade e guardrails heurísticos não garantem relevância ou segurança; respostas exigem validação humana. Verifique licenças e termos antes de uso ou redistribuição.

Na CLI, após o download, a preparação e a indexação são explícitas:

```
python scripts/prepare_data.py \
  --pubmedqa data/raw/pubmedqa.jsonl \
  --medquad data/raw/medquad.jsonl
python scripts/build_index.py --with-public-sources
MED_PUBLIC_RAG_ENABLED=true python main.py --user usuario-01
```